

torch.cuda.memory.memory_stats#

https://pytorch.org/docs/stable/generated/torch.cuda.memory.memory_stats.html

Description:

Return a dictionary of CUDA memory allocator statistics for a given device. The return value of this function is a dictionary of statistics, each of which is a non-negative integer. "allocated.{all,large_pool,small_pool}. {current,peak,allocated,freed}": number of allocation requests received by the memory allocator. "allocated_bytes.{all,large_pool,small_pool}. {current,peak,allocated,freed}": amount of allocated memory.

Function Signature

```
torch.cuda.memory.memory_stats(device=None)[source]#
```

Parameters

Return type

Returns:

dict[str, Any]

Notes & Warnings

NOTE

Note See Memory management for more details about GPU memory management.

NOTE

Note With backend:cudaMallocAsync, some stats are not meaningful, and are always reported as zero.

Detailed Documentation

Documentation

Return a dictionary of CUDA memory allocator statistics for a given device.

The return value of this function is a dictionary of statistics, each of which is a non-negative integer.

"allocated.{all,large_pool,small_pool}.{current,peak,allocated,freed}": number of allocation requests received by the memory allocator.

"allocated_bytes.{all,large_pool,small_pool}.{current,peak,allocated,freed}": amount of allocated memory.

"segment.{all,large_pool,small_pool}.{current,peak,allocated,freed}": number of reserved segments from cudaMalloc().

"reserved_bytes.{all,large_pool,small_pool}.{current,peak,allocated,freed}": amount of reserved memory.

"active.{all,large_pool,small_pool}.{current,peak,allocated,freed}": number of active memory blocks.

"active_bytes.{all,large_pool,small_pool}.{current,peak,allocated,freed}": amount of active memory.

"inactive_split.{all,large_pool,small_pool}.{current,peak,allocated,freed}": number of inactive, non-releasable memory blocks.

"inactive_split_bytes.{all,large_pool,small_pool}.{current,peak,allocated,freed}": amount of inactive, non-releasable memory.

For these core statistics, values are broken down as follows.

all: combined statistics across all memory pools.

large_pool: statistics for the large allocation pool (as of June 2025, for size $\geq$ 1MB allocations).

small_pool: statistics for the small allocation pool (as of June 2025, for size $<$ 1MB allocations).

current: current value of this metric.

peak: maximum value of this metric.

allocated: historical total increase in this metric.

freed: historical total decrease in this metric.

In addition to the core statistics, we also provide some simple event counters:

"num_alloc_retries": number of failed cudaMalloc calls that result in a cache flush and retry.

"num_ooms": number of out-of-memory errors thrown.

"num_sync_all_streams": number of synchronize_and_free_events calls.

"num_device_alloc": number of CUDA allocation calls. This includes both cuMemMap and cudaMalloc.

"num_device_free": number of CUDA free calls. This includes both cuMemUnmap and cudaFree.

The caching allocator can be configured via ENV to not split blocks larger than a defined size (see Memory Management section of the Cuda Semantics documentation). This helps avoid memory fragmentation but may have a performance penalty. Additional outputs to assist with tuning and evaluating impact:

"max_split_size": blocks above this size will not be split.

"oversize_allocations.{current,peak,allocated,freed}": number of over-size allocation requests received by the memory allocator.

"oversize_segments.{current,peak,allocated,freed}": number of over-size reserved segments from cudaMalloc().

The caching allocator can be configured via ENV to round memory allocations in order to reduce fragmentation. Sometimes the overhead from rounding can be higher than the fragmentation it helps reduce. The following stat can be used to check if rounding adds too much overhead:

"requested_bytes.{all,large_pool,small_pool}.{current,peak,allocated,freed}": memory requested by client code, compare this with allocated_bytes to check if allocation rounding adds too much overhead.

device (torch.device or int, optional) – selected device. Returns statistics for the current device, given by current_device(), if device is None (default).

See [Memory management](#) for more details about GPU memory management.

With backend:cudaMallocAsync, some stats are not meaningful, and are always reported as zero.